

Como se constrói um Sistema Operacional de Negócio

O mapa completo de um sistema que já roda de verdade — o que cada peça faz, por que ela existe, quanto custa e em que ordem construir. Escrito para quem vai decidir, com caixas técnicas para quem vai executar.

Referência: PMC OS (programa de aceleração) Leitura: ~25 min Agosto de 2026

O QUE TEM AQUI DENTRO

- | | |
|----------------------------------|---------------------------------------|
| 01 O que é um "OS de negócio" | 07 Onde a IA entra de verdade |
| 02 O prédio: as seis camadas | 08 Três jornadas ponta a ponta |
| 03 A fundação: o modelo de dados | 09 Em que ordem construir |
| 04 Duas portas: time e cliente | 10 Ferramentas e custo real |
| 05 Quem vê o quê (segurança) | 11 As armadilhas (aprendidas doendo) |
| 06 O encanamento: integrações | 12 As decisões que só você pode tomar |

O que é um "OS de negócio"

Não é um site. Não é um CRM. É o lugar onde a operação inteira acontece — e, principalmente, onde ela fica registrada.

Quase toda empresa de serviço vive espalhada: a lista de clientes está numa planilha, o histórico de atendimento está no WhatsApp, o agendamento está no Google Agenda, as metas estão na cabeça de alguém e o cliente não enxerga nada disso. Todo mundo trabalha, mas ninguém consegue responder rápido a perguntas simples: *quantos clientes estão ativos? quem não é atendido há 30 dias? esse cliente evoluiu ou não?*

Um sistema operacional de negócio resolve isso com três decisões, e é só isso que ele é:

- **Uma fonte da verdade.** Um único banco de dados onde o cliente existe uma vez só. Planilha, WhatsApp e agenda continuam existindo — mas viram *entradas*, não o registro oficial.
- **Duas portas para o mesmo dado.** Seu time entra por uma porta e vê tudo; o cliente entra por outra e vê só o que é dele. É o mesmo dado, com dois recortes.
- **Registro automático do que acontece.** Reunião, atendimento, entrega, tarefa: o sistema captura sozinho o que já ia acontecer de qualquer jeito, em vez de pedir que alguém digite depois.

O ganho não é "ter um sistema". É que a operação passa a ter memória — e memória é o que permite cobrar, renovar, provar resultado e escalar time sem perder qualidade.

O prédio: as seis camadas

Pense num prédio. Nenhuma camada é opcional, e elas se apoiam de baixo para cima: sem fundação, o resto racha. A ordem abaixo é literalmente a ordem em que

se constrói.



O sistema que serve de base aqui é uma aplicação React + TypeScript (Vite) publicada na Vercel, com Supabase como back-end único: PostgreSQL, autenticação, storage e funções serverless no mesmo lugar.

- **Front:** React 19, React Router, Tailwind + shadcn/ui, Recharts para gráficos, TanStack Query para cache de dados.
- **Back:** Postgres com `Row Level Security`, ~40 tabelas, funções `RPC` para leituras que precisam cruzar dados, e **Edge Functions** (Deno) para tudo que precisa de segredo — hoje são 24 delas.
- **Automação:** `pg_cron` chamando funções de hora em hora + n8n para os fluxos que envolvem ferramentas externas.
- **Deploy:** git push → Vercel. Migrations de banco versionadas em arquivos `.sql`, sempre com o par de *rollback*.

Nada aqui é obrigatório — mas essa combinação entrega camadas 1 a 3 quase de graça, e é por isso que ela é a recomendação padrão para o primeiro sistema de uma empresa.

A fundação: o modelo de dados

Se você só ler um capítulo, leia este. Aqui é onde o sistema se define.

Modelar dados é responder a uma pergunta chata: **quais são os substantivos do seu negócio?** Não os relatórios que você quer, não as telas — os substantivos. Em praticamente todo negócio de serviço, eles são seis:

SUBSTANTIVO	O QUE É	EXEMPLO NO DIA A DIA
Cliente	A empresa ou pessoa que contratou. Existe uma vez só, com um código próprio.	"Cliente 316 — DemoStore Brasil"
Pessoa	Gente ligada ao cliente ou ao seu time. Um cliente tem várias pessoas.	Dono, sócio, gerente, o consultor responsável
Atendimento	Um encontro que aconteceu ou vai acontecer, com hora, responsável e registro.	Reunião, visita, sessão, chamado
Tarefa	Algo combinado que tem dono e prazo. Sai do atendimento.	"Cliente vai enviar a planilha até sexta"
Indicador	Um número medido no tempo. É o que prova evolução.	Faturamento do mês, nota de satisfação, ganho gerado
Conteúdo	Material que o cliente consome dentro do sistema.	Aulas, modelos, trilhas, avisos

As três regras que evitam 80% da dor

- **Um cliente, um código, para sempre.** Um identificador curto, gerado pelo sistema, que nunca muda e aparece em tudo. Nome de empresa muda, CNPJ muda, e-mail muda — o código não. Sem isso, você acaba com o mesmo cliente cadastrado três vezes e relatórios que não fecham.
- **Nada de "campo texto livre" para coisa que vira número.** Status, tipo de plano, canal de venda: se o time digita à mão, você terá "Ativo", "ativo", "ATIVO " e "ativo (renovou)". Cada variação é uma linha a menos na sua métrica.
- **Data é data, não texto.** Guardar data como texto é o erro mais barato de cometer e mais caro de corrigir — some de relatório, quebra ordenação, e ninguém entende por quê.

Concretamente: chave primária técnica (`uuid` ou `bigint`) + um `codigo_cliente` humano gerado por *trigger* no insert. Datas em `timestampz` , sempre. Status em tabela de domínio ou `enum` , nunca `text` solto. Toda tabela nasce com `created_at` / `updated_at` .

Um padrão que vale a pena copiar: separar **tabela de entrada crua** (o que veio do formulário, sem limpeza, imutável) da **tabela operacional limpa** (o que o sistema usa). Quando o dado de origem vier torto — e vai vir —, você reprocessa sem perder o original.

Sobre "depois a gente ajusta": mudar uma tela custa horas. Mudar o modelo de dados quando já existem 300 clientes e 5.000 atendimentos custa semanas — e todo relatório histórico entra em suspeita. Vale gastar dois dias desenhando os substantivos antes de escrever a primeira tela.

04

Duas portas: time e cliente

Este é o desenho que define o produto: **um banco, duas experiências**. Não são dois sistemas, e não devem ser — dois sistemas viram dois cadastros e dois números diferentes para a mesma pergunta.

PORTA DO TIME

Lista de clientes, agenda geral, métricas globais, cadastros, permissões, relatórios de renovação

→

MESMO BANCO

Cliente · Pessoa · Atendimento · Tarefa · Indicador · Conteúdo

←

PORTA DO CLIENTE

Painel dele, histórico do atendimentos, tarefas combinadas, evolução c números, conteúdos liberados

O time enxerga N clientes; o cliente enxerga 1 — o dele. A diferença é uma regra no banco, não um sistema separado.

O que colocar em cada porta

A porta do time é sobre *gestão*: quem está em risco, quem não é atendido há tempo demais, qual consultor está sobrecarregado, quanto o programa gerou. A porta do cliente é sobre *valor percebido*: ele precisa abrir e enxergar em dez segundos o que ganhou, o que combinou e o que vem a seguir.

O erro comum é fazer a porta do cliente ser uma versão pobre da porta do time. Ela não é. Ela é a peça de retenção do seu negócio — é o que faz o cliente lembrar por que paga.

CAIXA TÉCNICA · COMO ISSO SE IMPLEMENTA

Uma única aplicação, um único roteador. Ao logar, o sistema decide o papel do usuário (o e-mail existe na tabela de time? então é interno) e monta o menu a partir disso. Rotas internas ficam atrás de um componente do tipo `<RequireSecao secao="clientes">`, que consulta as permissões daquele usuário.

Importante: **essa checagem no front é conveniência, não segurança**. A segurança real está no banco (próximo capítulo). O front esconde o menu; o banco recusa o dado.

05

Quem vê o quê

A pergunta que o dono deve fazer ao desenvolvedor: "se um cliente trocar o número na URL, ele vê o dado do concorrente?"

Na maioria dos sistemas feitos às pressas, a resposta é sim. A tela filtra o que mostra, mas o banco entrega tudo a quem pedir — e pedir é fácil.

A forma certa inverte isso: **a regra mora no banco**. Cada tabela ganha uma política do tipo "só devolva as linhas cujo cliente é o cliente de quem está perguntando". A partir daí, não importa por onde a pergunta venha — tela, aplicativo, script —, a

resposta já vem recortada. É a diferença entre um arquivo trancado e um arquivo aberto com um aviso de "não olhe".

Três papéis bastam para começar

- **Cliente** — vê exclusivamente os dados do próprio cadastro.
- **Time** — vê os clientes, com o menu recortado por seção (atendimento não precisa ver financeiro).
- **Dono/administrador** — vê tudo e administra acessos.

Permissão por seção — e não um cargo fixo por pessoa — é o que evita reescrever código toda vez que alguém muda de função. Você marca as seções que a pessoa abre numa tela de administração, e pronto.

CAIXA TÉCNICA · RLS NA PRÁTICA

No Postgres: `ALTER TABLE x ENABLE ROW LEVEL SECURITY` e políticas por operação. A política do cliente compara a identidade do token (`auth.uid()` / e-mail no JWT) com a coluna de dono da linha.

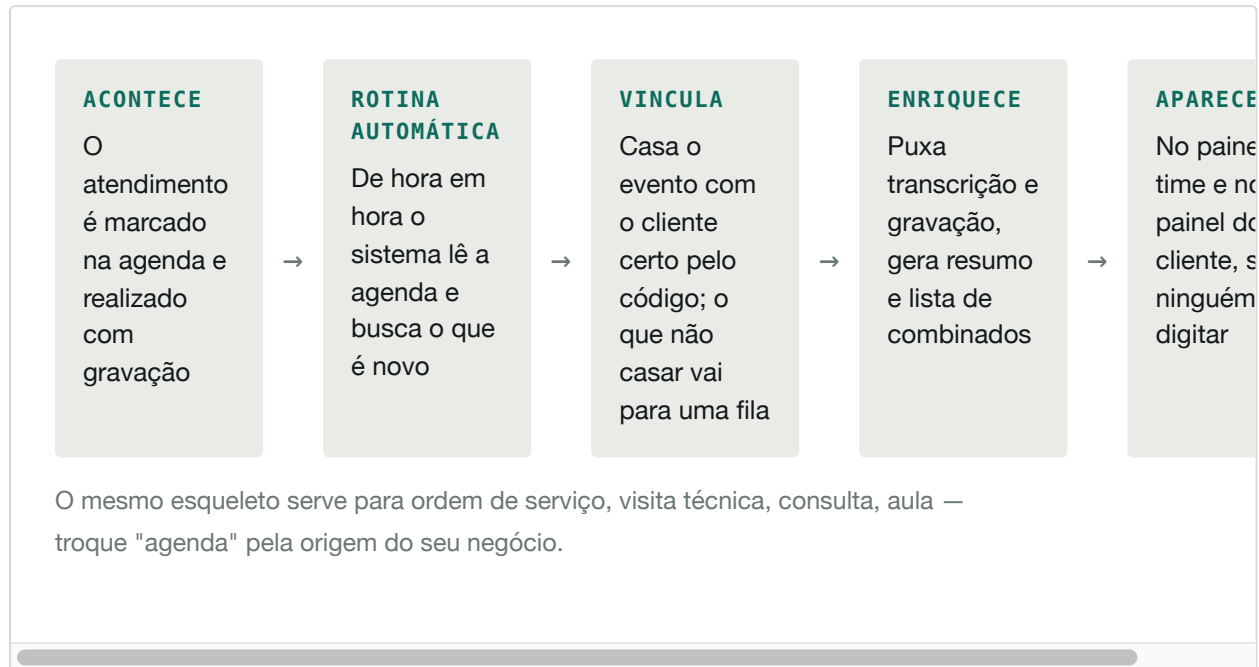
Três armadilhas reais desse desenho:

- RLS costuma ser escrita só para `SELECT`. Se o cliente precisa gravar algo (foto de perfil, resposta de formulário), ou você escreve a política de `UPDATE` ou expõe uma função `SECURITY DEFINER` específica — nunca abra a tabela inteira.
- Qualquer chave de serviço (*service role*) ignora RLS por definição. Ela só pode viver dentro de Edge Function ou script de servidor. Se ela aparecer no front, a segurança inteira caiu.
- Restringir permissão no banco **depois** que o front novo já estiver publicado. Fazer o contrário derruba o app em produção.

Teste de aceite antes de ir ao ar: logar como cliente A e tentar ler o cliente B — via tela e via chamada direta à API. As duas precisam devolver vazio.

O encanamento: integrações

Aqui está o pulo do gato de um sistema que as pessoas realmente usam: **ele captura o que já ia acontecer**, em vez de pedir digitação extra. Todo campo que depende de alguém lembrar de preencher vai estar vazio em três meses.



As integrações que quase sempre valem a pena

- **Agenda (Google/Outlook).** Fonte natural do que aconteceu e com quem. Vale nos dois sentidos: ler o que foi marcado e também criar eventos a partir do sistema.
- **Formulário de entrada.** Contrato assinado → formulário → cadastro criado sozinho, sem redigitação.
- **E-mail transacional.** Convite de acesso, link para definir senha, avisos. Nunca dependa de alguém mandar acesso na mão.
- **WhatsApp.** É onde o cliente já está. Comece por notificação de mão única (lembrete, confirmação); conversa bidirecional é um projeto à parte.
- **Transcrição de reunião.** Transforma uma hora de conversa em registro pesquisável — a matéria-prima de tudo que a IA fará depois.

Três mecanismos convivendo, e cada um cobre a falha do outro:

- **Webhook** para reagir ao instante (a agenda avisa que algo mudou);
- **Cron** de hora em hora (`pg_cron` chamando a função) para varrer a janela recente e pegar o que o webhook perdeu;
- **Watchdog** diário que verifica se a assinatura do webhook ainda está viva e a renova.

Toda sincronização precisa ser **idempotente**: rodar duas vezes não pode gerar duas linhas. Isso se resolve com uma chave única vinda da origem (o ID do evento) e `upsert`. E depois de inserir, confira que a linha existe mesmo — "processados: 12" no log não é prova de que gravou.

Credencial de integração fica em variável de ambiente da função, nunca no código nem no front. Para agenda corporativa, use conta de serviço com delegação, não o login de uma pessoa: pessoa sai da empresa, 2FA trava, e a automação morre.

07

Onde a IA entra de verdade

IA não é uma camada do sistema. É um acabamento que só funciona se as camadas de baixo estiverem cheias de dado bom.

Um chat inteligente sobre uma base vazia é um chat que inventa. Por isso a IA entra por último — e, quando entra, tem três usos que se pagam:

- **Transformar conversa em registro.** Pegar a transcrição de um atendimento e extrair resumo, combinados de cada lado e o resultado gerado. Economiza 20 a 40 minutos por atendimento e, principalmente, padroniza — dez pessoas diferentes registram do mesmo jeito.
- **Perguntar ao próprio sistema em português.** "Quais clientes estão sem atendimento há mais de 30 dias?" respondido em segundos, sem depender de alguém que saiba montar relatório.

- **Rascunhar o trabalho especializado.** Primeira versão de um plano, de um diagnóstico, de uma proposta — a partir dos dados que já estão lá. Rascunho, revisado por gente.

E onde ela não entra: decisão que tem consequência financeira ou contratual sem revisão humana; qualquer coisa que precise estar 100% correta sem ninguém conferir; e conversa com o cliente final sem supervisão, no começo.

CAIXA TÉCNICA · COMO PLUGAR SEM SE MACHUCAR

- A chave da IA fica na Edge Function, nunca no navegador. O front chama sua função; sua função chama o modelo.
- Para chat sobre dados, a função executa consulta **somente leitura** com a identidade do próprio usuário — assim a RLS continua valendo e o chat nunca vaza dado de outro cliente. É a diferença entre `SECURITY INVOKER` e `SECURITY DEFINER`: o segundo ignora as permissões de quem perguntou.
- Peça saída estruturada (JSON com esquema validado) sempre que o resultado for para o banco. Texto livre de modelo direto em coluna é dívida garantida.
- Trabalho longo trava a interface. Ou você transmite o resultado em tempo real (*streaming*), ou processa em fila e avisa quando ficar pronto — mas nunca deixe uma tela girando por dois minutos.
- Custo é por volume de texto. Resumir 200 atendimentos por mês custa dezenas de reais, não milhares — mas só depois de medir num lote pequeno.

08

Três jornadas ponta a ponta

A melhor forma de validar uma arquitetura antes de escrever código é narrar as jornadas críticas e ver se todas as peças aparecem naturalmente.

Jornada 1 — entra um cliente novo

Contrato assinado → link de formulário enviado → cliente preenche → cadastro criado automaticamente com código próprio → responsável interno é atribuído → o

cliente recebe um e-mail com link para criar a senha → ao entrar, encontra o painel dele já preenchido com o que informou, não uma tela vazia.

Esse último detalhe é o que decide a adoção. Painel vazio no primeiro acesso é a causa número um de cliente que nunca mais volta.

Jornada 2 — um atendimento acontece

Agendamento criado (pelo time ou pelo próprio cliente, num link público) → evento cai na agenda do responsável com o cliente convidado → o atendimento acontece e é gravado → a rotina automática identifica o evento, vincula ao cliente, traz transcrição e gravação → a IA extrai resumo, combinados e resultado → tudo aparece no histórico das duas portas, e as tarefas nascem já com dono e prazo.

Jornada 3 — o dono olha o painel na segunda de manhã

Abre e vê, sem clicar em nada: clientes ativos, atendimentos da semana, quem está sem contato há tempo demais, quem entra em renovação nos próximos 60 dias e o resultado acumulado gerado. Cada número é clicável e leva à lista que o compõe — número que não abre em lista não gera confiança nem ação.

Teste de arquitetura: se você não consegue narrar suas três jornadas críticas com as peças deste documento, falta alguma peça — ou tem peça sobrando. Narre antes de contratar o desenvolvimento; é a hora mais barata de descobrir.

Em que ordem construir

A sequência importa mais do que a velocidade. Cada fase abaixo entrega algo utilizável sozinha — se o projeto parar no meio, o que existe continua servindo.

FASE	O QUE ENTRA	COMO VOCÊ SABE QUE ACABOU	PRAZO TÍPICO
1. Fundação	Modelo de dados, login, papéis, cadastro de clientes, lista com busca e filtro	O time abandona a planilha de clientes	2–4 sem.
2. Operação	Registro de atendimentos, tarefas com dono e prazo, histórico por cliente	Dá para responder "o que aconteceu com esse cliente?" em 5 segundos	3–4 sem.
3. Porta do cliente	Acesso do cliente, painel dele, histórico, tarefas, evolução	Cliente entra sozinho e volta na semana seguinte	3–4 sem.
4. Automação	Agenda, formulário de entrada, e-mails automáticos, rotinas	Ninguém digita o que o sistema pode capturar	2–4 sem.
5. Inteligência	Resumo automático, chat sobre os dados, indicadores e alertas	O sistema avisa você antes de você perguntar	2–4 sem.

O que não fazer na fase 1

- **Aplicativo nativo.** Um site que funciona bem no celular resolve 95% do caso por 20% do custo.
- **Permissão granular por campo.** Três papéis bastam até você ter uns 15 usuários internos.
- **Relatório configurável pelo usuário.** Faça os cinco relatórios que você realmente olha. Construtor de relatório é um produto inteiro.
- **Integração com o ERP no primeiro mês.** Comece por exportação/importação de planilha; integre quando a dor for real.

- **Dois ambientes desde o dia um.** Um banco de desenvolvimento e um de produção, com o app escolhendo por variável de ambiente e um aviso visual quando não é produção. Testar em produção é o caminho mais curto para perder dado de cliente.
- **Mudança de banco é arquivo versionado.** Toda alteração de schema vira uma migration com data no nome — e com o par de rollback. Aplicar no dev, testar, depois aplicar em produção. Alterar tabela pelo painel gera diferença silenciosa entre os ambientes.
- **Backup verificado.** O provedor faz backup automático; o que quase ninguém faz é *restaurar* uma vez para conferir que funciona. Faça na fase 1, enquanto não dói.
- **Dado de cliente real não vai para o ambiente de teste.** Copie só tabelas de configuração e crie dois registros fictícios.

Ferramentas e custo real

Um sistema desse porte não tem custo de infraestrutura relevante. O custo é de construção e de manutenção — de gente, não de servidor.

PEÇA	FERRAMENTA DE REFERÊNCIA	ORDEM DE GRANDEZA / MÊS
Banco, login e funções	Supabase	Grátis para começar; ~US\$ 25 quando crescer
Publicação do site	Vercel	Grátis; ~US\$ 20 com time
Automações externas	n8n (nuvem ou próprio)	US\$ 0–25
E-mail transacional	Resend / SendGrid	Grátis até alguns milhares de envios
IA	API de modelo (por uso)	Dezenas de reais em uso normal
Construção	Desenvolvedor + IA de código	É aqui que está o custo real

Sobre construir com ferramentas de IA (Lovable, Claude Code e afins): elas aceleram muito as camadas 4 e 5 — telas, formulários, gráficos. Elas *não* substituem a decisão da camada 1. Chegar com o modelo de dados e as jornadas já desenhadas — como neste documento — é o que separa um sistema que dura de um protótipo bonito que precisa ser refeito no sexto mês.

Reserve orçamento de manutenção. Todo sistema vivo consome de 10% a 20% do custo de construção por ano só em ajuste, integração que muda do outro lado e pedido novo do time. Orçar só a construção é como comprar carro sem contar a gasolina.

As armadilhas (aprendidas doendo)

Cada item abaixo já custou tempo real em um sistema em produção. Leia como lista de perguntas para fazer a quem for construir o seu.

- **O mesmo cliente cadastrado duas vezes.** Duas linhas para a mesma empresa fazem o histórico se dividir e os relatórios discordarem entre si. Previna com

código único e busca por documento antes de todo cadastro.

- **Casar automaticamente pelo primeiro nome.** "Rodrigo" pode ser dois Rodrigues. Vinculação automática precisa de identificador forte; o que não casar vai para uma fila de revisão humana, não para um palpite.
- **Fuso horário e formato de data.** Um evento gravado com o fuso errado aparece no dia anterior. Padronize o formato na entrada e guarde sempre com fuso.
- **Campo obrigatório que ninguém preenche.** Se um dado só existe quando alguém lembra, ele não existe. Ou o sistema captura sozinho, ou o cadastro não avança sem ele.
- **Ordem errada ao apertar a segurança.** Restringir a permissão no banco antes do front novo estar publicado derruba a aplicação para todo mundo. Publique primeiro, restrinja depois.
- **Confiar no log em vez do banco.** "Importados: 30" não é prova. Depois de toda carga, confira contando as linhas.
- **Automação amarrada à conta de uma pessoa.** A pessoa sai, ativa verificação em duas etapas ou troca a senha, e a integração morre silenciosamente. Use conta de serviço e monitore.
- **Fazer tudo antes de mostrar algo.** Seis meses sem entrega é como um projeto morre. Ponha a fase 1 na mão do time e colha reclamação cedo — reclamação é informação de graça.

As decisões que só você pode tomar

Nenhum desenvolvedor consegue responder por você. Responda as doze antes de começar, por escrito — este é o briefing que vale mais do que qualquer especificação técnica.

- ☐ Quais são os **substantivos** do meu negócio? (a lista do capítulo 3, adaptada)
- ☐ O que exatamente é um "cliente" para mim: a empresa, a pessoa, ou o contrato?
- ☐ Quais são as **três perguntas** que hoje me custam mais tempo para responder?
- ☐ Meu cliente vai **entrar no sistema**? Se sim, o que ele precisa ver em dez segundos?

- ☐ Que dado já existe hoje e onde ele está? (planilha, agenda, WhatsApp, papel)
- ☐ Quem do meu time usa o sistema todo dia — e qual é a tela principal de cada um?
- ☐ Que evento do meu negócio vale a pena capturar automaticamente?
- ☐ Qual número prova o meu resultado para o cliente?
- ☐ Quantas pessoas do time entram, e o que cada perfil **não** pode ver?
- ☐ Qual é a fase 1 mínima que já substituiria uma planilha na segunda-feira?
- ☐ Quem, na minha empresa, é o dono do sistema depois de pronto?
- ☐ Quanto eu reservo por ano para manutenção e evolução?

Com essas doze respostas e as camadas deste documento, qualquer bom desenvolvedor — ou qualquer boa ferramenta de IA de construção — tem o que precisa. O que trava projeto raramente é a tecnologia; é a falta desse acordo escrito antes da primeira linha de código.

Documento de arquitetura de referência, baseado em um sistema em produção com dois portais, ~40 tabelas, 24 funções de servidor e sincronização automática de agenda. Escrito para ser adaptado, não copiado: a stack pode mudar; a ordem das camadas, não.